

Delegates in C#

Part-3

Anonymous Methods



Mr. Bangar Raju

 **NARESH**
technologies

Understanding Anonymous Methods in C#: A Comprehensive Guide

By Albert Harmon

Published April 25, 2017

🕒 4 min read

Education



TLDR This blog post explores anonymous methods in C#, explaining their definition, advantages, and practical applications through examples. It highlights how anonymous methods simplify code by allowing inline method definitions without

the need for named methods, while also discussing best practices for their use.

Welcome to this detailed exploration of anonymous methods in C#. In this post, we will delve into what anonymous methods are, how they relate to delegates, and their practical applications in programming.

What are Anonymous Methods?

Anonymous methods are a feature in C# that allows you to define a method without giving it a name. They are closely related to delegates, which are types that reference methods. In traditional usage, you would bind a named method to a delegate. However, with anonymous methods, you can bind a block of code directly to a delegate without needing a separate named method.

Recap of Delegates

Before we dive deeper into anonymous methods, let's briefly recap how delegates work:

1. **Define a Delegate:** You start by defining a delegate type that specifies the signature of the methods it can reference.
2. **Instantiate the Delegate:** You create an instance of the delegate and bind it to a method.
3. **Invoke the Delegate:** You can call the delegate, which in turn calls the bound method.

Implementing Anonymous Methods

To illustrate how anonymous methods work, let's consider a practical example. We will create a simple class that uses an anonymous method to greet a user.

Step 1: Define a Delegate

First, we define a delegate that will be used to call our greeting method:

```
public delegate string GreetingsDelegate(string name);
```

Step 2: Using a Named Method

In a traditional approach, we would create a method to greet the user:

```
public static string SomeGreetings(string name) {  
    return "Hello " + name + ", a very good morning!";  
}
```

Next, we instantiate the delegate and bind it to the named method:

```
GreetingsDelegate greetingsDelegate = new GreetingsDelegate(So  
string message = greetingsDelegate.Invoke("Scott");  
Console.WriteLine(message);
```



Step 3: Using an Anonymous Method

Now, let's simplify this by using an anonymous method. Instead of defining a separate method, we can directly define the logic inline:

```
GreetingsDelegate greetingsDelegate = delegate(string name) {  
    return "Hello " + name + ", a very good morning!";  
};  
string message = greetingsDelegate.Invoke("Scott");  
Console.WriteLine(message);
```



In this example, we used the `delegate` keyword to define an anonymous method that takes a string parameter and returns a greeting message. This eliminates the need for a separate method definition.

Advantages of Anonymous Methods

1. **Less Code:** Anonymous methods reduce the amount of code you need to write, as you can define the logic inline without creating a separate method.
2. **Improved Readability:** When the logic is small, it can be easier to read and understand since it is located right where it is used.
3. **No Need for Modifiers:** You do not need to specify access modifiers or return types, which simplifies the syntax.

When to Use Anonymous Methods

While anonymous methods can simplify your code, they are best used for small blocks of code (typically less than 10-12 lines). For larger methods, it is advisable to use named methods for better organization and maintainability.

Practical Application in Windows Forms

Let's see how anonymous methods can be applied in a Windows Forms application. Suppose we want to dynamically create a button that shows a message when clicked:

1. **Create a Button:** We add a button to the form and handle its click event.
2. **Use an Anonymous Method:** Instead of creating a separate event handler method, we can use an anonymous method directly:

```
button.Click += delegate(object sender, EventArgs e) {  
    MessageBox.Show("Hello World");  
};
```

This approach allows us to define the click event logic inline, making the code cleaner and easier to follow.

Conclusion

Anonymous methods in C# provide a powerful way to simplify code by allowing inline method definitions. They are particularly useful for small code blocks where creating a separate method would be unnecessary. However, it is essential to use them judiciously, as they can lead to less readable code if overused. By understanding when and how to use anonymous methods, you can enhance your programming efficiency and maintainability.

Thank you for reading! For more insights into C# and programming concepts, stay tuned for our upcoming posts.